

État de l'art

Jumeau numérique d'une infrastructure de calcul

Prénom Nom

September 27, 2026

Contents

1	Introduction	3
1.1	Contexte et objectifs	3
1.2	Méthodologie de recherche	3
1.3	Plan de l'état de l'art	3
2	Le Cloud Computing : origines, forces et limites	4
2.1	Origines et évolution	4
2.2	Points forts du Cloud Computing	4
2.3	Points faibles et limites	5
2.4	Vers le jumeau numérique : pourquoi ce concept émerge	5
3	Simulateurs d'infrastructures cloud et data center	6
3.1	Panorama	6
3.2	Fiches de lecture	6
3.2.1	CloudNetSim++ [8]	6
3.2.2	CloudSim [4]	6
3.2.3	GreenCloud [6]	7
4	Jumeaux numériques appliqués au cloud et au réseau	8
4.1	Définition et périmètre	8
4.2	Fiches de lecture	8
4.2.1	Network Digital Twin pour le continuum Edge-Cloud [2]	8
4.2.2	Jumeaux numériques pour data centers (simulation, conception, contrôle thermique) [7]	8
4.2.3	Jumeau numérique d'infrastructure d'entreprise (Cyber-Range-as-a-Service) [1]	8
5	Infrastructure as Code et simulation/prédiction	10
5.1	Panorama	10
5.2	Terraform	10
5.2.1	Présentation et fonctionnement	10
5.2.2	Points forts	11
5.2.3	Limites	11
5.2.4	Pertinence pour le jumeau numérique	11
5.2.5	Infracost [3]	11
6	Prédiction de performance	12
6.0.1	CloudProphet [5]	12
6.0.2	Modèles ML de prédiction de charge cloud [9]	12

7	Prédiction de coûts et FinOps	13
7.1	Panorama	13
7.1.1	Cloud FinOps [10]	13
8	Synthèse et positionnement	14
8.1	Tableau comparatif	14
8.2	Constats	14
8.3	Positionnement du sujet	14
9	Conclusion	15

Chapter 1

Introduction

1.1 Contexte et objectifs

Ce document présente l'état de l'art réalisé dans le cadre du sujet de tutorat « *Jumeau numérique d'une infrastructure de calcul* ». L'objectif est d'explorer les clouds et leurs évolutions dans le contexte du *Fog Computing* (cloud public, privé, hybride, multi-cloud, DevOps, SRE), afin de déterminer dans quelle mesure un jumeau numérique peut permettre la simulation et la prédiction (coûts/FinOps, performance) dans la formation et l'optimisation d'une infrastructure de calcul.

1.2 Méthodologie de recherche

- **Bases consultées** : IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, arXiv, Google Scholar.
- **Mots-clés** : *digital twin cloud, network digital twin, cloud simulator, data center simulation, infrastructure as code simulation, Terraform cost prediction, FinOps forecasting, cloud performance prediction machine learning*.
- **Période couverte** : 2010–2026.
- **Critères d'inclusion** : travaux proposant un outil, un modèle ou une méthode de simulation/prédiction pour une infrastructure de calcul (cloud, data center, edge), ou portant sur le concept de jumeau numérique appliqué à l'IT.

1.3 Plan de l'état de l'art

Le document est organisé en six axes, correspondant aux dimensions du sujet : un rappel sur le cloud computing, ses origines et ses limites qui motivent le concept de jumeau numérique (chapter 2), les simulateurs d'infrastructures cloud/datacenter (chapter 3), les jumeaux numériques appliqués au cloud/réseau (chapter 4), l'infrastructure as code — en particulier Terraform — et son lien avec la simulation (chapter 5), la prédiction de performance (chapter 6) et la prédiction de coûts / FinOps (chapter 7). Une synthèse comparative et un positionnement du sujet concluent le document (chapter 8).

Chapter 2

Le Cloud Computing : origines, forces et limites

2.1 Origines et évolution

Le *cloud computing* s'appuie sur des technologies de virtualisation apparues dès la fin des années 1990 (hyperviseurs de type VMware), qui permettent de faire tourner plusieurs machines virtuelles isolées sur un même serveur physique. Le tournant commercial a lieu au milieu des années 2000 : Amazon lance *Elastic Compute Cloud (EC2)* et *Simple Storage Service (S3)* en 2006, suivis par Google App Engine (2008) et Microsoft Azure (2010), popularisant un modèle de ressources informatiques louées à la demande plutôt qu'achetées et exploitées en interne.

En 2011, le NIST (*National Institute of Standards and Technology*) publie une définition de référence du cloud computing autour de cinq caractéristiques essentielles (libre-service à la demande, accès réseau large, mutualisation des ressources, élasticité rapide, service mesuré), de trois modèles de service (IaaS, PaaS, SaaS) et de quatre modèles de déploiement (cloud public, privé, communautaire, hybride).

Plus récemment, l'essor de l'*Internet des objets* et des applications sensibles à la latence (temps réel, véhicules autonomes, usines connectées) a fait émerger le *Fog Computing* et l'*Edge Computing* : une partie du traitement est rapprochée de la source de données plutôt que centralisée dans un data center distant, donnant naissance à des architectures multi-niveaux (device – edge – fog – cloud) et à la notion de *multi-cloud* (utilisation simultanée de plusieurs fournisseurs). En parallèle, les pratiques *DevOps* et *SRE (Site Reliability Engineering)* se généralisent pour automatiser le déploiement, l'exploitation et la fiabilisation de ces infrastructures de plus en plus complexes.

2.2 Points forts du Cloud Computing

- **Élasticité et scalabilité** : possibilité d'ajuster dynamiquement les ressources allouées (CPU, mémoire, stockage) en fonction de la charge réelle.
- **Modèle économique à la consommation (pay-as-you-go)** : suppression des coûts d'investissement matériel (CapEx) au profit de coûts opérationnels (OpEx) proportionnels à l'usage.
- **Accessibilité et mise à disposition rapide** : provisionnement de ressources en quelques minutes, contre plusieurs semaines pour du matériel physique.
- **Résilience et haute disponibilité** : réplification géographique, tolérance aux pannes, mécanismes de reprise après sinistre proposés nativement par les fournisseurs.

- **Externalisation de la maintenance** : le fournisseur cloud prend en charge la maintenance matérielle, les mises à jour de sécurité de l'infrastructure sous-jacente, etc.

2.3 Points faibles et limites

- **Imprévisibilité des coûts** : la facturation à l'usage rend difficile l'anticipation précise de la dépense, en particulier pour des architectures complexes ou des pics de charge non planifiés — d'où l'émergence de la discipline FinOps (chapter 7).
- **Complexité croissante** : la multiplication des services managés, des régions et des fournisseurs (multi-cloud) rend l'infrastructure difficile à appréhender dans son ensemble, y compris pour les équipes qui l'exploitent.
- **Dépendance au fournisseur (*vendor lock-in*)** : l'utilisation de services propriétaires complique la portabilité vers un autre fournisseur.
- **Difficulté à tester avant déploiement** : les environnements de production réels (Amazon EC2, etc.) limitent l'expérimentation, et il est risqué/coûteux de tester des changements d'infrastructure ou des algorithmes d'ordonnancement directement en production [8].
- **Sécurité et confidentialité** : mutualisation des ressources physiques entre plusieurs clients (multi-tenant), surface d'attaque élargie, questions de conformité réglementaire selon la localisation des données.
- **Consommation énergétique** : les data centers représentent une part importante et croissante de la consommation électrique mondiale, avec des enjeux de refroidissement et d'empreinte carbone [8].

2.4 Vers le jumeau numérique : pourquoi ce concept émerge

Les limites précédentes ont un point commun : elles proviennent toutes d'un manque de **visibilité et de prévisibilité** avant que les changements ne soient réellement appliqués à l'infrastructure. Un opérateur ne peut aujourd'hui connaître avec certitude, avant déploiement, ni le coût exact d'une configuration, ni son comportement sous charge, ni l'impact d'une modification de topologie ou d'un changement de fournisseur.

C'est précisément ce manque que le concept de **jumeau numérique** (*digital twin*) vise à combler : construire une réplique virtuelle, continuellement synchronisée avec l'infrastructure réelle, permettant de *simuler* et de *prédire* son comportement (coût, performance, consommation énergétique) avant, pendant et après un changement, sans risquer l'environnement de production. Le chapitre suivant (chapter 4) détaille ce concept et les travaux existants qui s'en approchent.

Chapter 3

Simulateurs d'infrastructures cloud et data center

3.1 Panorama

3.2 Fiches de lecture

3.2.1 CloudNetSim++ [8]

Problème traité Les simulateurs cloud existants manquent de modèles réseau réalistes (au niveau paquet) et d'interface graphique riche pour l'analyse fine.

Approche Simulateur au niveau paquet construit sur OMNeT++/INET, avec des modules dédiés : *VM Manager*, *User Task Scheduler*, module d'énergie, gestion de SLA et de tarification, support multi-site (data centers géographiquement distribués) et topologies mesh/star/ring.

Validation Vérification formelle par réseaux de Petri de haut niveau (HLPN) et solveur SMT/Z3 ; évaluation expérimentale (usage CPU/mémoire, délai de construction de la GUI, scalabilité utilisateurs, consommation énergétique) sur une charge réelle de 22 385 tâches.

Limites Pas d'ajout dynamique de serveurs pendant la simulation ; topologies inter-datacenter limitées à trois types ; pas de synchronisation bidirectionnelle avec une infrastructure réelle (donc pas un « vrai » jumeau numérique au sens strict).

Pertinence Référence centrale pour la partie *simulation d'infrastructure* : bon exemple de modularité (VM Manager, scheduler, module énergie) réutilisable pour la conception du jumeau numérique visé par ce projet. Le Tableau 8.1 reprend sa comparaison avec 6 autres simulateurs (iCanCloud, CloudSim, GreenCloud, CloudSched, MDCSim, CloudAnalyst).

3.2.2 CloudSim [4]

Problème traité

Approche VM/hôtes/politiques d'allocation, pas de modèle réseau fin.

Limites

Pertinence

3.2.3 GreenCloud [6]

Problème traité

Approche énergie.

Limites

Pertinence

Chapter 4

Jumeaux numériques appliqués au cloud et au réseau

4.1 Définition et périmètre

4.2 Fiches de lecture

4.2.1 Network Digital Twin pour le continuum Edge–Cloud [2]

Problème traité Absence de synthèse centrée sur les spécificités du jumeau numérique réseau (NDT) dans le contexte Edge-Cloud Continuum.

Approche Revue de littérature couvrant l’architecture, les composants clés du NDT, l’intégration de modèles de machine learning/IA, la gestion de la virtualisation et les capacités du Software-Defined Networking.

Limites

Pertinence Cadre de référence pour définir l’architecture générale (collecte de données, modélisation, boucle de contrôle) du jumeau numérique du projet.

4.2.2 Jumeaux numériques pour data centers (simulation, conception, contrôle thermique) [7]

Problème traité Panorama des solutions existantes de jumeau numérique appliquées aux data centers.

Approche Catégorisation en plateformes de simulation physique et en architectures cloud/edge (ex. Azure Digital Twins, AWS IoT TwinMaker) ; met en avant la synchronisation avec des capteurs et systèmes de gestion réels.

Limites Les solutions purement cloud présentent des problèmes de latence pour le contrôle temps réel.

Pertinence Illustre le compromis edge/cloud à considérer pour l’architecture du prototype.

4.2.3 Jumeau numérique d’infrastructure d’entreprise (Cyber-Range-as-a-Service) [1]

Problème traité

Approche Réplication d’une infrastructure d’entreprise (utilisateurs, ressources, trafic réseau) via un modèle statistique, instanciée sur des ressources cloud à grande échelle.

Limites

Pertinence graphique de l'infrastructure (sujet connexe mentionné dans la fiche de tutorat).

Chapter 5

Infrastructure as Code et simulation/prédiction

5.1 Panorama

L'*Infrastructure as Code* (IaC) consiste à décrire une infrastructure (serveurs, réseaux, bases de données, permissions...) sous forme de fichiers texte versionnables plutôt que de la configurer manuellement via une interface graphique. Cette description devient alors une source de vérité exploitable par des outils tiers : c'est cette propriété qui rend l'IaC particulièrement intéressante comme **point d'entrée** d'un jumeau numérique, puisqu'elle fournit déjà, sous forme structurée, l'état désiré de l'infrastructure.

5.2 Terraform

5.2.1 Présentation et fonctionnement

Terraform est un outil open source d'Infrastructure as Code développé par HashiCorp, publié en 2014. Il permet de décrire, de manière **déclarative**, l'état souhaité d'une infrastructure dans des fichiers écrits en *HashiCorp Configuration Language* (HCL), puis de faire converger l'infrastructure réelle vers cet état via un cycle de vie en trois étapes :

- **terraform init** : initialise le projet et télécharge les *providers* nécessaires (modules qui savent dialoguer avec l'API d'un fournisseur cloud donné : AWS, Azure, Google Cloud, OVH, etc.) ;
- **terraform plan** : calcule et affiche le **différentiel** entre l'état actuel de l'infrastructure (stocké dans un fichier d'état, le *state*) et l'état décrit dans le code, sans rien modifier réellement — cette étape est particulièrement intéressante pour un jumeau numérique, car elle constitue déjà une forme de « simulation » de ce qui *va* se passer ;
- **terraform apply** : applique effectivement les changements calculés à l'étape précédente pour faire converger l'infrastructure réelle vers l'état désiré.

Terraform est **multi-cloud** par construction : le même langage HCL permet de décrire des ressources chez différents fournisseurs (voire plusieurs simultanément), ce qui en fait un candidat naturel pour représenter des architectures hybrides ou multi-cloud comme celles évoquées dans le sujet de ce projet.

5.2.2 Points forts

- **Déclaratif et idempotent** : on décrit l'état voulu (pas la suite d'actions pour y arriver), et appliquer plusieurs fois le même code produit le même résultat.
- **Prévisualisation avant action (plan)** : réduit le risque de changement destructif non anticipé.
- **Multi-cloud et multi-fournisseur** via un système extensible de *providers* et un large écosystème communautaire.
- **Versionnable** : le code d'infrastructure peut être suivi dans Git au même titre que du code applicatif (revue de code, historique, rollback).

5.2.3 Limites

- **Gestion du fichier d'état (*state*)** : ce fichier décrit la correspondance entre le code et les ressources réelles ; sa perte, sa corruption ou une divergence non détectée (*drift*, un changement fait manuellement en dehors de Terraform) peuvent désynchroniser la représentation et la réalité — un problème central pour un jumeau numérique qui doit justement rester synchronisé avec l'infrastructure réelle.
- **Pas de prédiction de coût ni de performance native** : `terraform plan` indique *quelles ressources* vont changer, mais ne dit rien sur le coût résultant ni sur le comportement sous charge — d'où le recours à des outils complémentaires comme Infracost (ci-dessous) pour le volet coût.
- **Courbe d'apprentissage du langage HCL** et complexité des modules pour de grandes infrastructures.
- **Risque d'actions destructives** : une mauvaise compréhension du *plan* peut mener à la suppression accidentelle de ressources en production.

5.2.4 Pertinence pour le jumeau numérique

Le triplet *code déclaratif* – *state* – *plan* de Terraform présente une analogie directe avec l'architecture générale d'un jumeau numérique (modèle – état courant – simulation d'un changement avant application). Le fichier de state peut être vu comme une première brique de synchronisation entre le monde réel et sa représentation virtuelle, et la sortie de `terraform plan` comme une entrée exploitable pour un module de prédiction de coût (type Infracost) ou, à terme, de performance.

5.2.5 Infracost [3]

Problème traité Absence de visibilité sur le coût d'une infrastructure avant son déploiement effectif.

Approche Analyse statique des fichiers Terraform (ou du plan Terraform), association de chaque ressource déclarée aux données de tarification du fournisseur cloud, intégration en CI/CD (commentaire de coût sur chaque pull request).

Limites Estimation basée sur des hypothèses d'usage pour les services à la consommation (ex. Lambda, S3) ; pas de simulation du comportement réel de l'infrastructure (charge, latence, etc.), seulement du coût.

Pertinence Brique *FinOps* directement réutilisable / comparable pour le module de prédiction de coût du jumeau numérique visé.

Chapter 6

Prédiction de performance

6.0.1 CloudProphet [5]

Problème traité Prédiction de performance d'applications s'exécutant dans des VM « boîte noire » en cloud public, rendue difficile par l'absence de visibilité sur le contenu de la VM.

Approche Identification préalable de l'application exécutée dans la VM, puis sélection de métriques runtime fortement corrélées comme entrée d'un modèle de machine learning pour prédire le niveau de performance ; introduction d'un indice de dégradation de performance pour les charges variables.

Résultats Amélioration de plus de $2\times$ par rapport aux méthodes de prédiction existantes en termes d'erreur maximale.

Limites jeu d'applications connu, généralisation à des charges inédites, etc.)

Pertinence Exemple de méthode ML de prédiction de performance exploitable pour le module « prédiction » du jumeau numérique.

6.0.2 Modèles ML de prédiction de charge cloud [9]

Problème traité Difficulté d'estimer précisément l'usage des ressources du fait de la variabilité et de la dimensionnalité des charges de travail cloud.

Approche Classification et taxonomie de cinq catégories d'approches de prédiction, comparaison expérimentale sur trois traces de charge réelles.

Limites

Pertinence Vue d'ensemble utile pour choisir la famille de modèles (statistique, ML classique, deep learning) adaptée au module de prédiction du jumeau numérique.

Chapter 7

Prédiction de coûts et FinOps

7.1 Panorama

7.1.1 Cloud FinOps [10]

Problème traité Absence de discipline structurée pour gérer financièrement un modèle de dépense cloud variable.

Approche Cadre méthodologique en trois phases (Inform, Optimize, Operate) ; couvre la prévision de dépenses, l'allocation des coûts et les stratégies de remise sur engagement.

Limites Ouvrage de référence pratique plutôt qu'une contribution scientifique/technique ; peu de détails algorithmiques sur la prévision.

Pertinence Cadre conceptuel pour définir les indicateurs FinOps que le jumeau numérique devra prédire (coût par ressource, dérive budgétaire, etc.).

Chapter 8

Synthèse et positionnement

8.1 Tableau comparatif

Table 8.1: Comparaison synthétique des travaux étudiés

Outil / Travail	Type	Modèle réseau	Énergie	Prédiction perf.	Prédiction coût	Sync. temps réel	Open source
CloudNetSim++	Simulateur	Réel (paquet)	Oui	Non	Non	Non	Oui
CloudSim	Simulateur	Non	Oui	Non	Non	Non	Oui
GreenCloud	Simulateur	Paquet	Oui	Non	Non	Non	Oui
Infracost	Outil IaC	–	–	Non	Oui	Non	Oui
CloudProphet	Modèle ML	–	–	Oui	Non	Non	% TODO
NDT (survey)	Concept / archi.	Variable	Variable	Variable	Variable	Oui (visé)	–
Jumeau numérique visé (ce projet)	Jumeau numérique	À définir	À définir	Oui	Oui	Oui	À définir

8.2 Constats

8.3 Positionnement du sujet

Chapter 9

Conclusion

Bibliography

- [1] A next-generation platform for cyber range-as-a-service, 2021. arXiv:2112.11233, <https://arxiv.org/pdf/2112.11233>.
- [2] A comprehensive survey of network digital twin architecture, capabilities, challenges, and requirements for edge–cloud continuum. *Computer Networks (ScienceDirect)*, 2025. <https://www.sciencedirect.com/science/article/abs/pii/S014036642500101X>.
- [3] Infracost — cloud cost estimation for terraform, 2026. <https://www.infracost.io>.
- [4] Rodrigo N. Calheiros, Rajiv Ranjan, Anton Beloglazov, César A. F. De Rose, and Rajkumar Buyya. Cloudsim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. *Software: Practice and Experience*, 41(1):23–50, 2011.
- [5] Darong Huang, Luis Costero, Ali Pahlevan, Marina Zapater, and David Atienza. Cloudprophet: A machine learning-based performance prediction for public clouds. 2023. arXiv:2309.16333, <https://arxiv.org/pdf/2309.16333>.
- [6] Dzmitry Kliazovich, Pascal Bouvry, Yury Audzevich, and Samee U. Khan. Greencloud: A packet-level simulator of energy-aware cloud computing data centers. In *Proc. IEEE Global Telecommunications Conference (GLOBECOM)*, pages 1–5, 2010.
- [7] Jung-Hua Liu. Digital twin solutions for data centers: Simulation, system design, and thermal control, 2025. Medium, <https://medium.com/@gwrx2005/digital-twin-solutions-for-data-centers-simulation-system-design-and-thermal-control-dff>.
- [8] Asad W. Malik, Kashif Bilal, S. U. Malik, Zahid Anwar, Khurram Aziz, Dzmitry Kliazovich, Nasir Ghani, Samee U. Khan, and Rajkumar Buyya. Cloudnetsim++: A gui based framework for modeling and simulation of data centers in omnet++. *IEEE Transactions on Services Computing*, 10(4):506–519, 2017.
- [9] Deepika Saxena, Ashutosh Kumar Singh, and Rajkumar Buyya. Performance analysis of machine learning centered workload prediction models for cloud. *IEEE Transactions on Parallel and Distributed Systems*, 2023. <https://arxiv.org/pdf/2302.02452>.
- [10] J. R. Storment and Mike Fuller. *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management*. O’Reilly Media, 2nd edition, 2023.